

FINDER: APLICACIÓN DE OPTIMIZACIÓN NO LINEAL PARA LA RESOLUCIÓN DE PROBLEMAS DE PROCESAMIENTO DE LENGUAJE NATURAL

Paola Rodríguez
Laura Pamplona
David Sánchez

I. DESCRIPCIÓN DEL PROBLEMA

El análisis de similitud semántica entre textos complejos representa un desafío importante en el procesamiento del lenguaje natural. Muchos textos contienen relaciones y significados profundos que no pueden ser comprendidos por enfoques que se limitan a comparar palabras o patrones superficiales. Esto dificulta que los sistemas identifiquen cuando dos textos expresan ideas similares utilizando distintas formas de redacción.

II. INTRODUCCIÓN

El crecimiento acelerado de la información textual en entornos digitales ha impulsado el desarrollo de métodos automáticos que faciliten su análisis y comprensión. Si bien existen múltiples enfoques dentro del campo del procesamiento del lenguaje natural (PLN), persisten limitaciones al momento de representar con precisión la similitud semántica entre distintos textos. En este contexto, el presente proyecto propone un enfoque que proyecta los textos en un espacio vectorial y construye grafos que reflejan sus relaciones semánticas. Esta representación matemática permite agrupar textos con significados afines y visualizar las conexiones entre ellos, lo cual resulta útil para tareas de clasificación y análisis automatizado.

III. OBJETIVOS

III-A. Objetivo General

Utilizar la teoría de grafos y la representación vectorial de textos como herramientas para analizar y visualizar la similitud semántica entre descripciones personales. Al modelar estas relaciones en forma de grafo, se busca identificar patrones de cercanía con-

ceptual, comprender cómo se agrupan los textos según su significado y descubrir conexiones semánticas que podrían no ser evidentes a simple vista.

III-B. Objetivos Específicos

- Recolectar un conjunto de descripciones personales que sirvan como base de datos para el modelamiento y análisis de similitud semántica.
- Implementar un proceso de preprocesamiento y vectorización de textos mediante embeddings, garantizando la preservación de relaciones semánticas entre los términos.
- Modelar las relaciones entre descripciones personales como un grafo ponderado, donde los nodos representen textos y las aristas indiquen el grado de similitud entre ellos.
- Desarrollar visualizaciones que permitan interpretar los resultados del grafo, identificando agrupaciones semánticas y relaciones no evidentes entre las descripciones.

IV. MARCO TEÓRICO

IV-A. Antecedentes

La similitud semántica entre textos, ha sido un tema que ha sido explorado al hablar sobre el procesamiento del lenguaje natural, en sus inicios, la metodología con que se abordaba tradicionalmente era en enfoques léxicos o estadísticos como lo eran la frecuencia de términos o la representación de bolsa de palabras, los cuales eran modelos que consideraban los textos como una serie de palabras sin ningún contexto, el problema que tenían era que contaban con ciertas limitaciones al no poder capturar directamente relaciones semánticas profundas entre palabras.

Con el desarrollo del deep learning definido por Jim Holdsworth (s.f) como "subconjunto del machine learning que utiliza redes neuronales multicapa, llamadas redes neuronales profundas, para simular el complejo poder de toma de decisiones del cerebro humano" se crearon los modelos de embeddings, como:

- **Word2Vec:** Fue el pionero, este usa una red neuronal con 3 capas, una de entrada, una oculta y una de salida, la cual se entrena con muchos textos para ajustar los vectores de las palabras, haciendo que las predicciones sean correctas según el contexto dado.

Este modelo es muy bueno porque captura eficazmente las propiedades semánticas y aritméticas de una palabra" (DataScientist, 2022) y porque abrió las puertas a muchos otros modelos como GloVe y BERT.

- **GloVe:** Está diseñado para capturar las relaciones semánticas entre palabras analizando la frecuencia con la que aparecen en un mismo contexto.

La forma en la que funciona es generar un pre-procesamiento dividiendo los textos en palabras individuales, posteriormente se crea un vocabulario con la frecuencia con la que aparece una palabra, luego se construye una matriz de coocurrencia en la que se cuenta la frecuencia con la que aparece la palabra cerca a otras palabras en un contexto dado y si dos palabras generalmente ocurren juntas o pegadas, los vectores serán cercanos dentro del espacio de embedding.

- **BERT:** al igual que Word2Vec y GloVe, transforma las palabras en vectores de un espacio continuo, preservando las relaciones semánticas existentes entre ellas. Este modelo por su parte, utiliza el contexto bidireccional para generar una mejora en el rendimiento en las tareas de procesamiento de lenguaje natural.

Estos modelos fueron un gran avance debido a que ya no solo se podía comparar textos en función de su coincidencia literal, si no también en función del significado.

Un ejemplo claro del uso de los embeddings es Tinder, actualmente esta aplicación trabaja con una inteligencia artificial que procesa el lenguaje a través del modelo Word2Vec. Al identificar la similitud que hay dentro de los patrones del lenguaje, se les permite a los usuarios conectar más con personas que se les asemejen, por el comportamiento y las preferencias denotadas en el lenguaje.

En el contexto del procesamiento del lenguaje natural, los grafos han sido implementados para modelar relaciones entre documentos y distintas estructuras, trabajar con grafos en ese tipo de contextos implica la visualización de estructuras de datos complejas y al usar grafos ponderados, es visualmente más fácil reconocer conexiones entre textos a partir de los pesos asignados a las aristas según su respectiva similitud.

Enfoques como los del presente proyecto, demuestran que las representaciones vectoriales combinadas con los grafos permite capturar la proximidad conceptual entre textos.

IV-B. Bases Teóricas

La teoría de grafos es una rama de las matemáticas con aplicaciones dentro de las ciencias de la computación, en donde se estudian estructuras compuestas por vértices y aristas que los conectan. En el contexto de este proyecto, los grafos permiten representar textos como nodos y las relaciones de similitud semántica entre ellos como aristas ponderadas, lo que facilita la visualización y el análisis de la cercanía conceptual entre diferentes descripciones personales.

- **Procesamiento del lenguaje natural:** Permite a los dispositivos reconocer, generar y comprender textos usando aprendizaje automático, facilitando la comunicación con el lenguaje humano.
- **Embeddings:** Representaciones vectoriales de palabras o frases en un espacio continuo, donde se comprenden relaciones semánticas entre palabras.
- **Espacio vectorial:** Estructura matemática formada por vectores donde la suma y la multiplicación escalar están definidas. Permite representar textos como vectores en $\mathbb{R}^n$.
- **Distancia euclidiana:** Representa el camino más corto entre dos puntos: $d(a, b) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$. Entre menor sea la distancia, mayor es la similitud entre textos.
- **Similitud semántica:** Mide qué tan similares son dos textos en significado más allá de coincidencias literales.
- **Grafos ponderados:** Estructuras donde las aristas tienen un peso asociado. En este proyecto, ese peso representa el grado de similitud entre descripciones personales.

V. MODELAMIENTO DEL PROBLEMA

El modelamiento del problema propuesto por FINDER parte de la necesidad de representar de manera estructurada las relaciones semánticas existentes entre distintos textos o descripciones personales. Con el fin de capturar las similitudes contextuales y cuantificar el grado de relación entre los textos, se utiliza una combinación de *teoría de grafos* y *representaciones vectoriales* obtenidas mediante técnicas de procesamiento del lenguaje natural (PLN).

V-A. Representación del conjunto de textos

Se considera un conjunto de representaciones de corpus de texto enumeradas T , donde T es un subconjunto del conjunto universal de textos TU . Cada elemento de T corresponde a una descripción o documento que será analizado. A partir de este conjunto se declara el **Grafo** $G(T)$, que servirá como modelo formal de las relaciones semánticas entre los textos.

La definición matemática del grafo se da de la siguiente manera:

$$\{n_i\}_{i=1}^{|T|} = K$$

$$N(G(T)) = K$$

$$E(G(T)) = \{(a, b) \in K \times K \mid a \neq b\}$$

Asimismo, se define la función:

$$te = \{(n_i, t_i) \in K \times T\}$$

la cual asigna a cada nodo del grafo su respectivo texto.

Ejemplo gráfico

A continuación, se presenta un ejemplo de grafo $G(T)$ para un caso concreto, en el cual el conjunto T contiene tres descripciones personales en distintos idiomas. Cada nodo representa un texto, y la función te asocia a cada uno su correspondiente descripción.

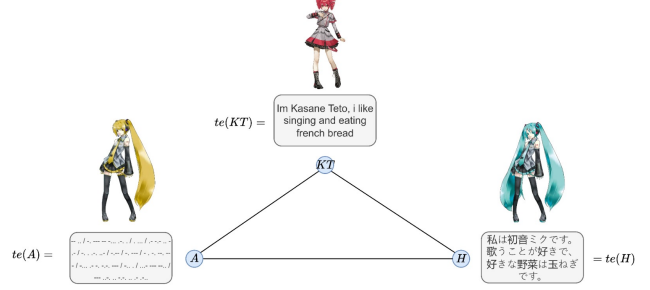


Figura 1: Grafo representativo del modelamiento de tres entidades con sus descripciones textuales.

El grafo presentado representa la relación semántica entre tres descripciones personales escritas en distintos idiomas. Cada nodo del grafo corresponde a un texto específico y las aristas reflejan el grado de similitud entre ellos, calculado mediante la distancia euclidiana entre sus representaciones vectoriales.

Algoritmo propuesto

V-B. Función de vectorización

Con el fin de cuantificar el contexto de cada texto, se introduce la función:

$$Vec : T \rightarrow \mathbb{R}^n$$

que asigna a cada corpus un vector en un espacio n -dimensional. Cada componente del vector representa características semánticas derivadas del contenido del texto, obtenidas mediante modelos de *embeddings* o representaciones distribuidas.

Esta función nos permite comparar textos escritos en distintas lenguas dentro de un mismo espacio vectorial. De este modo, textos con significados conceptualmente similares se ubicarán a distancias cortas entre sí, independientemente del idioma en que estén redactados.

Por ejemplo, dos textos que hablen sobre *cine* tendrán una distancia euclidiana menor entre sus vectores que un texto sobre *cine* y otro sobre *deportes*.

V-C. Construcción del grafo semántico

La estructura de grafo propuesta tiene como objetivo representar las similitudes contextuales entre los textos del conjunto T a través de las aristas. Cada arista conecta dos nodos y se etiqueta con un valor que cuantifica el grado de semejanza entre los contextos de los textos que representa.

De manera formal, el peso de la arista E_{ab} se define mediante una métrica de similitud basada en la distancia euclidiana entre los vectores de los textos:

$$d(a, b) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

$$sim(a, b) = \frac{1}{1 + d(a, b)}$$

De esta forma, cuanto más pequeños sean los valores de $d(a, b)$, mayor será la similitud entre los textos asociados, y viceversa. El conjunto de pesos $sim(E_{ab})$ proporciona una medida cuantitativa que permite interpretar y visualizar las relaciones semánticas entre los textos.

El procedimiento general del algoritmo es el siguiente:

1. Iterar sobre los nodos n_i y vectorizar los textos asociados: $Vec(t_i)$.
2. Para cada par de nodos (a, b) , calcular la distancia euclidiana $dist_{euclid}(Vec(a), Vec(b))$.
3. Asociar el valor calculado a la arista con extremos (a, b) .

El primer paso puede implementarse mediante un recorrido en profundidad o en anchura sobre el grafo, dependiendo de la estrategia elegida.

[12pt]article [utf8]inputenc graphicx float amsmath amssymb caption

VI. SOLUCIÓN PROPUESTA

La solución planteada consiste en desarrollar, en Python, un sistema capaz de evaluar la similitud semántica entre descripciones personales mediante técnicas de procesamiento del lenguaje natural y teoría de grafos. Para ello, los textos se preprocesan y se transforman en vectores utilizando un modelo de *embeddings*, lo que permite representar su contenido semántico de manera cuantificable.

Con estos vectores se construye un grafo donde cada nodo corresponde a un texto y cada arista indica el grado de similitud entre dos descripciones. A partir de esta estructura se analizan patrones, agrupamientos y relaciones relevantes dentro del conjunto de datos. Finalmente, se genera una visualización del grafo que facilita la interpretación de la red semántica y se formulan conclusiones sustentadas en los resultados obtenidos.

VII. IMPLEMENTACIÓN DE LA SOLUCIÓN

La implementación del sistema FINDER se desarrolló utilizando el lenguaje de programación Python,

junto con diversas librerías orientadas al procesamiento del lenguaje natural, análisis de similitud semántica y construcción de grafos. A continuación, se detalla exhaustivamente cada una de las etapas que conforman el proceso de modelamiento.

VII-A. Preparación del entorno de trabajo

Para garantizar un rendimiento adecuado, se verificó la disponibilidad de una GPU mediante la librería **torch**. Esta aceleración permitió procesar de manera eficiente el modelo de generación de embeddings. Asimismo, se instalaron las siguientes librerías:

- **Unsloth**: utilizada para cargar el modelo de embeddings Qwen2 7B de forma optimizada.
- **NetworkX**: empleada para la construcción, manipulación y análisis del grafo semántico.
- **Pandas**: utilizada para la lectura y limpieza del archivo CSV.
- **PyForceAtlas2**: responsable de generar la disposición espacial del grafo.
- **Matplotlib**: utilizada para la visualización final del grafo.

Este conjunto de herramientas permitió consolidar un entorno robusto para el procesamiento del corpus.

VII-B. Carga y depuración del conjunto de textos

La base de datos fue cargada mediante el módulo `files.upload()` y posteriormente procesada con **pandas**. Durante este proceso, se eliminaron entradas inválidas como `[deleted]` o textos vacíos. El conjunto final de descripciones se almacenó en la lista **textos**.

Se construyó además el diccionario:

$$te = \{"texto_i : t_i\}$$

que asocia cada descripción con un identificador único. Este diccionario permitió recuperar los textos desde sus nodos correspondientes dentro del grafo.

VII-C. Construcción del grafo semántico

Se creó un grafo vacío mediante **NetworkX** y se añadieron nodos para cada elemento del diccionario **te**. Posteriormente, se generó un **grafo completo** mediante la adición de una arista entre cada par de nodos distintos:

$$E = \{(a, b) \mid a \neq b, a, b \in N(G)\}$$

De esta forma se garantizó que todas las descripciones fueran comparadas entre sí.

VII-D. Vectorización mediante embeddings

Cada descripción fue transformada en un vector numérico utilizando el modelo Qwen2 7B Embedding a través de la librería Unsloth. Para mantener coherencia en la inferencia, se empleó una plantilla que integra el texto dentro de una instrucción:

Instruct: I Corpus: t_i

A continuación, se generó la representación vectorial normalizada para cada nodo mediante:

$$Vec(t_i) = \frac{v_i}{\|v_i\|_2}$$

donde v_i es el embedding crudo producido por el modelo.

VII-E. Cálculo de similitud semántica

Una vez generados los vectores de cada texto, se procedió a calcular la distancia semántica entre cada par de nodos del grafo. Para cada arista (a, b) se obtuvo primero la distancia euclidiana entre los embeddings:

$$d(a, b) = \|\text{Vec}(a) - \text{Vec}(b)\|_2$$

A partir de dicha distancia se definió un valor de peso, utilizado únicamente con fines visuales para representar cuán similares o diferentes son dos textos. Este peso se calculó mediante:

$$w(a, b) = \frac{1}{0,5 + d(a, b)}$$

Es importante señalar que:

- Un valor alto de $w(a, b)$ indica textos semánticamente cercanos.
- Un valor bajo de $w(a, b)$ indica textos semánticamente muy distintos.
- Estos valores no determinan la posición de los nodos en la visualización, sino únicamente el grosor de las aristas.

Los pesos calculados se almacenaron en la lista **values**, que posteriormente se usó como parámetro **width** al dibujar el grafo, permitiendo distinguir visualmente la fuerza semántica entre pares de textos.

VII-F. Disposición espacial mediante ForceAtlas2

Para obtener una representación visual más comprensible del grafo completamente conectado, se empleó el algoritmo ForceAtlas2. Este método genera una disposición espacial basada exclusivamente en fuerzas de atracción y repulsión entre nodos, sin considerar los valores de similitud previamente calculados.

Por tanto, es fundamental aclarar que:

ForceAtlas2 no utiliza la similitud semántica para ubicar los nodos.

Su propósito es únicamente proporcionar una visualización equilibrada que evite la superposición entre nodos en grafos densos.

El resultado del algoritmo se almacenó en un diccionario:

`pos = ForceAtlas2_layout( $G$ )`

el cual contiene las coordenadas en dos dimensiones para cada nodo del grafo.

VII-G. Visualización final del grafo

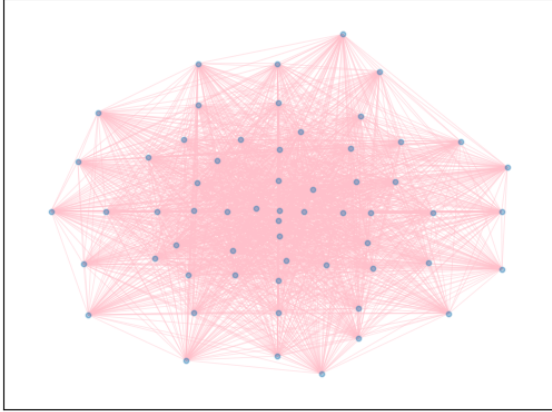
Con las posiciones generadas y los pesos calculados, se procedió a la visualización del grafo mediante **networkx**. La configuración utilizada fue:

- **node_size = 10**: nodos pequeños para evitar saturación visual.
- **alpha = 0.4**: transparencia en las aristas.
- **width = values**: grosor proporcional al peso semántico.
- **with_labels = False**: ocultar etiquetas para mayor claridad.

Esta visualización permite identificar:

- Aristas gruesas: textos similares.
- Aristas delgadas: textos muy diferentes.

[H]



Visualización final del grafo con ForceAtlas2.

VII-H. Acceso a la información del grafo

El diccionario `te` permitió acceder al contenido original asociado a cada nodo. Esto fue especialmente útil tras identificar los pares de textos con pesos extremos.

Dado que los textos `texto_11` y `texto_46` presentaron el menor peso semántico dentro del grafo, se procedió a inspeccionar su contenido para validar su diferencia conceptual mediante:

```
print(te["texto_11"])
print(te["texto_46"])
```

```
print(te["texto_11"])
... I might be a very unusual guy. My hobbies and interests
    I'm just a honest, and lonely guy who distracts himself
```

Figura 2: Contenido del nodo `texto_11`.

```
print(te["texto_46"])
... I know it's getting real tough for you now but *60* hour work weeks AND a full
    I failed out JC while working less than part time and volunteering (weekly). M
    &nbsp;
    &nbsp;
    I had some small trouble on how to start this. You and most people start with
    28 IT "support" guy that has dull and kind of secluded life interspersed with
    Like you and many people who have commented (and internet citizens in general)
    &nbsp;
    Random but I miss hard liquor. Been drinking beer recently and, while the taste
    I'm pretty involved in the alt-right movement, strengthening immigration laws,
```

Figura 3: Contenido del nodo `texto_46`.

La revisión manual confirmó que ambos textos difieren ampliamente en temática, estructura y carga descriptiva, coherente con su baja similitud vectorial.

VIII. RESULTADOS OBTENIDOS

El sistema permitió construir exitosamente un grafo semántico totalmente conectado a partir de los textos procesados. Cada par de nodos fue enlazado mediante un valor de peso calculado según la distancia entre sus embeddings.

Los resultados mostraron una distribución amplia de pesos:

- Pesos altos: relaciones semánticas estrechas.
- Pesos bajos: textos conceptualmente alejados.

El análisis reveló que el par formado por:

(`texto_11`, `texto_46`)

corresponde al menor nivel de similitud semántica. La inspección directa de sus contenidos confirmó esta diferencia, lo cual valida el modelo y el procedimiento implementado.

La visualización con ForceAtlas2 permitió obtener un diagrama legible del grafo, aunque su disposición es estética y no semántica. Sin embargo, el grosor de las aristas sí permitió resaltar patrones relevantes, como:

- regiones con aristas gruesas (textos similares),
- regiones con aristas delgadas (textos muy distintos).

Estos resultados reflejan correctamente la estructura semántica interna del corpus utilizado.

IX. CONCLUSIONES

El proyecto FINDER demostró que la combinación de embeddings semánticos y teoría de grafos permite representar y analizar de manera efectiva las relaciones conceptuales entre textos.

Los vectores generados por el modelo capturaron la esencia semántica del corpus, y el uso de un grafo ponderado permitió cuantificar y visualizar estas relaciones de forma clara. La corrección conceptual aplicada durante la revisión aseguró una interpretación precisa de los resultados.

La identificación del par con menor similitud semántica confirmó la capacidad del sistema para distinguir textos con temáticas muy diferentes. Asimismo, la visualización del grafo permitió apreciar diferentes patrones.

En conjunto, FINDER constituye una herramienta válida y robusta para estudiar la similitud semántica, abriendo oportunidades para análisis temáticos, clasificación o agrupamiento en proyectos futuros.

X. BIBLIOGRAFÍA

- Stryker Cole, Holdsworth Jim (s.f). ¿Qué es el PLN (procesamiento del lenguaje natural)?. IBM. <https://www.ibm.com/es-es/think/topics/natural-language-processing>
- Pablo Huet (12 de julio de 2024). Embeddings: Què son y còmo transforman datos en informacìon. OpenWebinars. <https://openwebinars.net/blog/embeddings/>
- EBSCO (s.f). espacio vectorial. EBSCO. <https://www.ebsco.com/research-starters/physics/vector-space>
- Chugani Vinod (1 de octubre de 2024). Comprender la distancia euclidiana: De la teoría a la práctica. <https://www.datacamp.com/es/tutorial/euclidean-distance>
- NLP Cloud (s.f). API de similitud semántica. <https://nlpccloud.com/es/nlp-semantic-similarity-api.html>
- Attenborough Maria (2003). Teoría de grafos. ScienceDirect. <https://www.sciencedirect.com/topics/computer-science/weighted-graph>
- Zertuche Oriol (2024). Los 8 mejores modelos de incrustación de texto en 2024.cody.<https://meetcody.ai/es/blog/los-8-mejores-modelos-de-incrustacion-de-texto-en-2024/>
- Holdsworth Jim (s.f). ¿Qué es el deep learning?. IBM. <https://www.ibm.com/es-es/think/topics/deep-learning>
- DataScientist (2022). Word2Vec: NLP Word Embedding. DataScientist. <https://datascientest.com/es/nlp-word-embedding-word2vec-es>
- Kaur Gurneet (2023). The Unexpected Love Affair: How AI Transforms Tinder's Dating Experience?. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2023/05/the-unexpected-love-affair-how-ai-transforms-tinders-dating-experience/>
- Geeks for Geeks (2025). Glove Word Embedding in NLP. <https://www.geeksforgeeks.org/nlp/Glove-Word-Embedding-in-NLP/>
- Kashyap Kathrani (2020). All about embeddings. Medium. <https://medium.com/@kashyapkathrani/all-about-embeddings-829c8ff0bf5b>